

Relatório de Criptografia e Segurança em redes

Licenciatura em Engenharia de
Telecomunicações e Informática

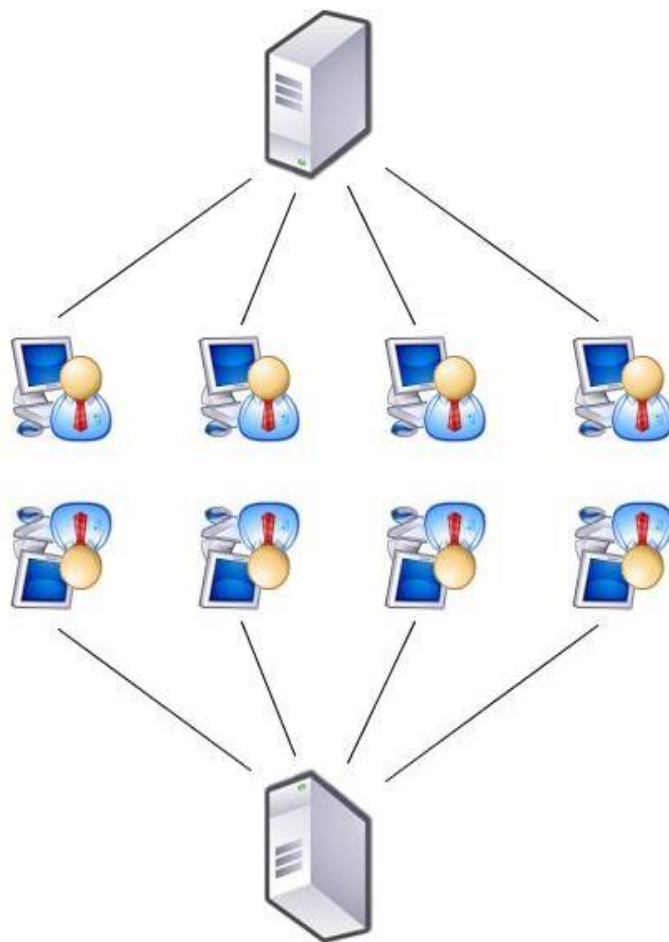
2024/2025

Da autoria de:

Bernardo Salgado A103664

Luís Baptista A105419

Luís Marques A103674



Universidade do Minho

Índice

1.	Introdução	3
2.	Fluxograma de ideias	4
3.	Arquitetura do Sistema de Comunicação Segura	5
4.	Explicação do código	8
5.	Discussão de Resultados e Conclusão	10
6.	Bibliografia.....	11

1. Introdução

Este trabalho tem como objetivo desenvolver um sistema assíncrono e seguro para a troca de mensagens entre utilizadores, composto por um servidor e múltiplos clientes.

- O sistema visa oferecer um **ambiente robusto** onde a **confidencialidade, integridade e autenticidade das mensagens são garantidas**.
- O servidor desempenha o papel de intermediário, armazenando temporariamente as mensagens enviadas até que os destinatários as solicitem.
- Cada cliente possui um identificador único que permite o encaminhamento correto das mensagens.
- Estas mensagens são estruturadas com campos essenciais, como **id_msg**, **id_origem**, **id_destinatário**, **assunto**, **conteudo** e **timestamp**, garantindo a organização na comunicação. (Linha 16-20 no ficheiro mensagem.py)

Para assegurar um elevado nível de segurança, utilizamos criptografia assimétrica (RSA), protegendo a confidencialidade das mensagens, e assinaturas digitais, que asseguram a integridade e autenticidade das informações.

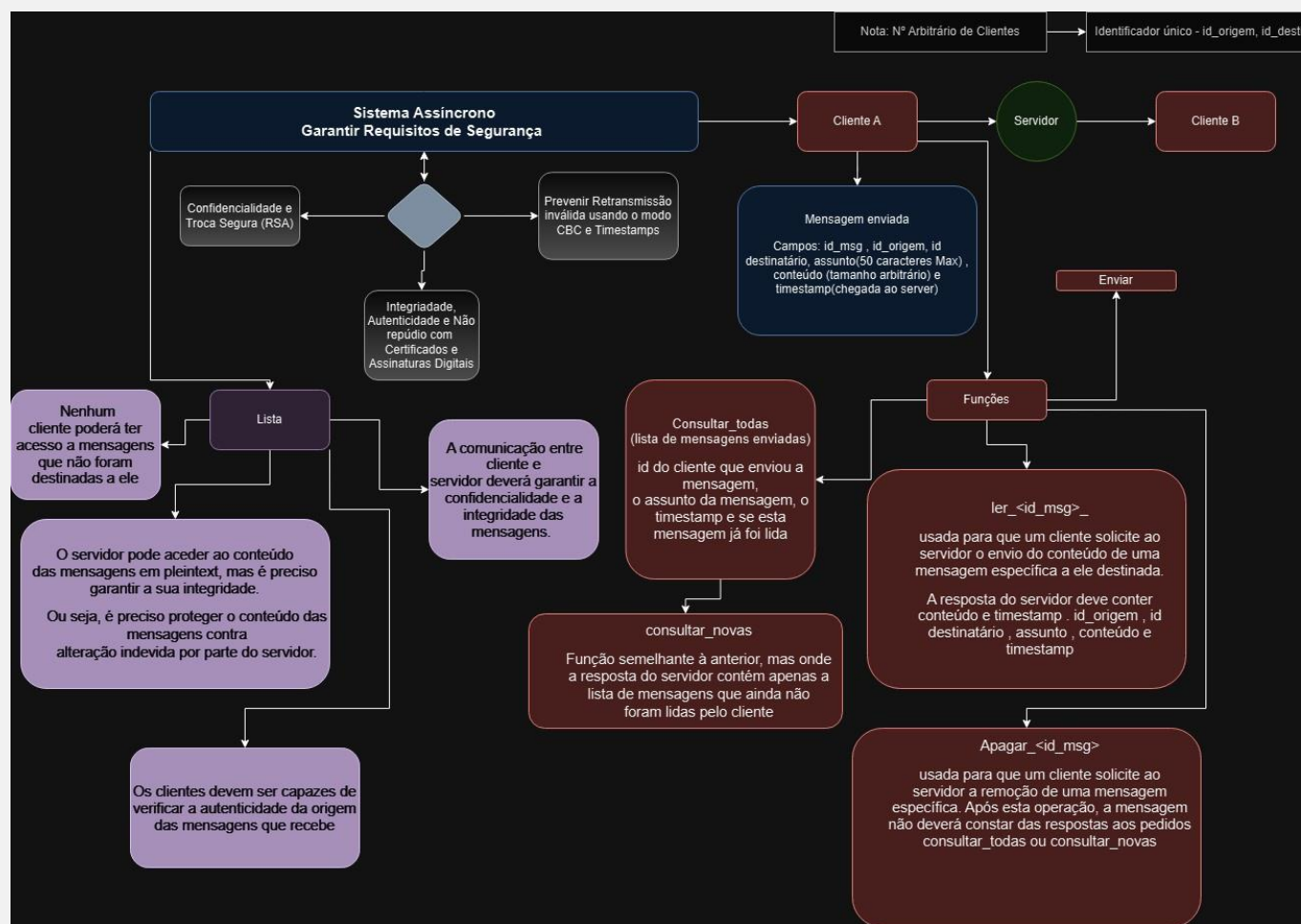
- Este relatório apresenta uma implementação prática das funcionalidades desenvolvidas, com foco na segurança de troca de mensagens e na facilidade de utilização pelos utilizadores.

O trabalho pretende, assim, apresentar um sistema que equilibre segurança, eficiência e simplicidade na gestão de mensagens assíncronas.

2. Fluxograma de ideias

Neste Fluxograma de ideias, simplificamos os objetivos do trabalho, de modo a facilitar a sua orientação e execução

Imagem 1



3. Arquitetura do Sistema de Comunicação Segura

Componentes Principais:

1. Cliente (Client.py):

O cliente é responsável por se comunicar com o servidor de forma segura, enviando e recebendo mensagens criptografadas. (Linha 79-94, 97-99 no ficheiro server.py)

- Certificados e chave privada RSA: Cada cliente possui um certificado digital e uma chave privada RSA para autenticação. (Ficheiros estão armazenados na diretoria /certs)

Processo de Envio e Recebimento de Mensagens:

- O cliente valida o certificado do servidor. (Linha 91-98 no ficheiro client.py)
- Gera uma chave publica a partir da chave privada RSA. (Linha 100-105 no ficheiro client.py)
- Criptografa as mensagens usando AES e envia-as ao servidor. (Linha 166-176 no ficheiro client.py)
- Recebe mensagens criptografadas e as descriptografa com a chave AES. (Linha 205-210 no ficheiro client.py)

2. Servidor (Server.py):

O servidor valida as mensagens dos clientes e controla as mensagens enviadas e recebidas, oferecendo segurança através da criptografia e autenticação.

O servidor tem o seu próprio certificado e chave privada RSA. (Ficheiros armazenados na diretoria /certs)

Processo de Comunicação:

- Valida o certificado do cliente. (Linha 66-71 no ficheiro server.py)
- Gera uma chave AES para criptografar e descriptografar as mensagens. (Linha 88 no ficheiro server.py)
- Criptografa a chave AES com a chave pública do cliente. (Linha 89-96 no ficheiro server.py)
- Recebe as mensagens criptografadas, descriptografa-as usando AES, e processa as ações solicitadas (como "get_messages", "send_message", etc.). (Linha 109-265 no ficheiro server.py)
- Armazena as mensagens de forma segura, associadas ao destinatário.

3. Mensagens:

As mensagens são representadas por um objeto Mensagem (formato - JSON) com os campos id_msg, id_origem, id_destinatario, assunto, conteúdo e timestamp. (Linha 4-33 no ficheiro mensagem.py)

Criptografia de Mensagens: A criptografia de mensagens é feita com o algoritmo AES em modo CBC, e a chave AES é protegida com criptografia RSA. (Linha 41-48 no ficheiro server.py e linha 124-131 no ficheiro client.py)

4. Certificados e Chaves:

- Chaves RSA: São usadas para a troca segura de chaves AES e autenticação mútua entre cliente e servidor.
- Certificados X.509: São gerados para cada cliente e servidor para garantir a autenticidade e integridade das comunicações. (Linha 17-46 no ficheiro cert_generator.py)

5. Fluxo de Comunicação:

Como Gerar Chaves e Certificados?

O servidor gera suas chaves e certificado, e o mesmo processo é feito para cada cliente. (Presente no ficheiro cert_generator.py)

As chaves privadas são armazenadas de forma segura, enquanto os certificados são usados para autenticar as partes.

- Autenticação: O servidor autentica os clientes através dos seus certificados digitais e vice-versa. (Linha 66-71 no ficheiro server.py e linha 91-98 no ficheiro client.py)
- Troca de Chave: O cliente gera e envia a chave pública RSA para o servidor, que a usa para criptografar a chave AES. (Linha 100-109 no ficheiro client.py)
- Envio de Mensagens: O cliente envia mensagens criptografadas com a chave AES. O servidor recebe, descriptografa e processa as mensagens conforme necessário. (Linha 174-190 no ficheiro client.py)
- Ações de Mensagens: O servidor pode realizar ações como "marcar mensagens como lidas", "enviar mensagem", "apagar mensagem", e "ver mensagens novas", todas com criptografia e autenticação. (Linha 209-265 no ficheiro server.py)

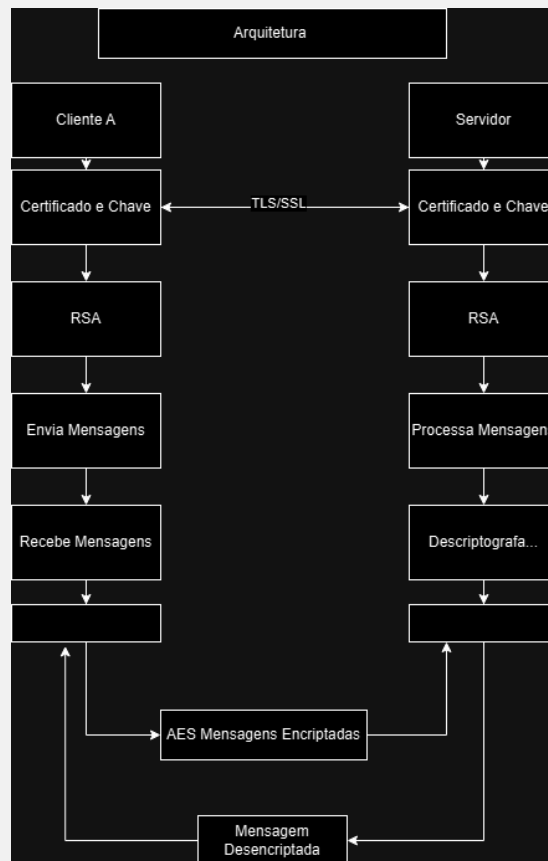


Imagem 2

- Segurança com Certificados e Criptografia RSA: Cada cliente e o servidor possuem certificados digitais X.509 e chaves privadas RSA para garantir autenticidade e troca segura de chaves. (Presentes na diretoria /certs e gerados pelo ficheiro cert_generator.py)
- Criptografia Simétrica (AES): A chave AES é gerada para criptografar as mensagens, garantindo confidencialidade e eficiência na troca de mensagens.
- Gestão de Mensagens: O servidor administra a recepção, armazenamento e envio das mensagens de acordo com as ações solicitadas pelo cliente. (Função que começa na linha 209 no ficheiro Server.py)
- Validação de Certificados e Chaves: O servidor e o cliente autenticam mutuamente usando certificados digitais e validação de chaves públicas.
- Esta arquitetura garante a comunicação segura entre cliente e servidor com criptografia forte (RSA e AES) e autenticação mútua, tornando o sistema adequado para o envio e recebimento de mensagens de forma privada e segura.

4. Explicação do código

O sistema implementado visa garantir a comunicação segura entre cliente e servidor utilizando criptografia. O servidor e os clientes geram os seus próprios pares de chaves RSA e certificados digitais X.509 para autenticação.

- A chave pública RSA é utilizada para criptografar uma chave AES, que é usada para criptografar as mensagens trocadas entre as partes.
- O servidor oferece funcionalidades como enviar e receber mensagens, além de ações como marcar mensagens como lidas, apagar mensagens e ver mensagens novas.
- Cada comunicação é criptografada para garantir que apenas o destinatário autorizado possa lê-la.

Principais componentes do código:

- **Criar Chaves e Certificados:** Geração de pares de chaves RSA e certificados digitais X.509 para autenticação. (Presente no ficheiro `cert_generator.py`)
- **Troca de Chaves e Criptografia:** Uso da RSA para trocar uma chave AES, que é usada para criptografar as mensagens. (Linha 84-95 no ficheiro `Server.py`)
- **Ações do Servidor:** O servidor lida com o envio, recebimento, marcação e exclusão de mensagens de maneira segura. (Linha 209-265 no ficheiro `Server.py`)
- **Segurança da chave e do certificado:** Uso de palavra-chave para proteger as chaves privadas e os certificados dos diferentes clientes. Assim assegura a integridade e segurança da comunicação. (Linha 28 no ficheiro `Client.py`)
- **Uso do Modo CBC no AES:** Usado para criptografia simétrica (Linha 127 no ficheiro `client.py`, e linha 44 no ficheiro `Server.py`)

Uso do Modo CBC com AES

- Garante **confidencialidade** ao misturar padrões no texto cifrado.
- E ainda maior segurança com o uso de um vetor de inicialização (IV).

Para que serve? -> Criptografar e descriptografar mensagens trocadas entre cliente e servidor.

Uso do AES

- É **rápido e eficiente** para criptografar grandes quantidades de dados
- Oferece alta segurança contra ataques

Para que serve? -> Proteger o conteúdo das mensagens trocadas entre cliente e servidor

Uso do RSA

- Permite a **troca segura de chaves AES**, mesmo sem um segredo compartilhado previamente.
- Oferece autenticação mútua ao validar certificados e chaves públicas.

Para que serve? -> Garantir a troca segura da chave AES e validar a identidade do cliente e do servidor por meio de certificados X.509.

Integração de RSA e AES

- RSA protege a troca inicial da chave AES, garantindo confidencialidade.
- AES é usado para criptografar mensagens devido à sua eficiência e segurança.

Para que serve? -> Proporcionar um canal seguro e eficiente para troca e armazenamento de mensagens.

5. Discussão de Resultados e Conclusão

Os testes realizados mostraram que o sistema funcionou corretamente, com a criptografia e a troca de chaves sendo realizadas com sucesso.

- A autenticação via certificados digitais foi eficaz, garantindo que apenas clientes autorizados se podem ligar ao servidor.
- A criptografia AES, que foi usada para a comunicação das mensagens, proporcionou um bom desempenho sem afetar a segurança.
- O sistema foi resistente a tentativas de intercepção, pois as mensagens eram protegidas com criptografia robusta e a troca de chaves AES foi realizada de forma segura usando RSA.

A gestão das mensagens, incluindo as funcionalidades de exclusão e marcação como lidas, foi implementada corretamente e funcionou sem problemas durante os testes. O sistema oferece uma comunicação segura e eficiente, combinando RSA para a troca de chaves e AES para a criptografia das mensagens.

Durante os testes, a autenticação e criptografia funcionaram corretamente, e o servidor foi capaz de gerir as mensagens de maneira eficiente. Em termos de segurança, o sistema demonstrou ser robusto, protegendo as mensagens contra intercepção e modificação.

O desempenho também foi satisfatório, com a criptografia não afetando negativamente a velocidade da comunicação. Este sistema é adequado para aplicações que exigem troca segura de mensagens!

6. Bibliografia

Imagens:

- Imagem da capa: <https://arqserv.wordpress.com/2012/03/17/como-funciona-a-arquitetura-cliente-servidor/>
- Logotipo UM - <https://mobfood.pt/empresas/um-universidade-do-minho/>
- Imagem 1 - <https://app.diagrams.net/>
- Imagem 2 - <https://app.diagrams.net/>

Sites:

- Slides das aulas teóricas (professor João Marco Silva) disponibilizados em - [https://elearning.uminho.pt/ultra/courses/ 62249 1/outline](https://elearning.uminho.pt/ultra/courses/62249_1/outline)

Documentação:

- <https://www.cs.vsb.cz/ochodkova/courses/kpb/cryptography-and-network-security-principles-and-practice-7th-global-edition.pdf>
- <https://cryptography.io/en/latest/>
- <https://docs.openssl.org/master/>
- [Symmetric encryption — Cryptography 44.0.0.dev1 documentation](#)